Chapter 3 – Design of the Overall Application

3.1 Overall Design Purpose

In our implementation, Chapter 3 mainly explains how Task A and Task B were connected into one complete Java program. Task A is responsible for reading the three candidate datasets, comparing three sorting algorithms, and selecting the Top 10 highest-priority locations from each dataset. Task B then uses these selected locations as important inspection targets and calculates the required shortest paths on the weighted graph.

In the final version of the project, the program is designed as a console-based Java application. It does not use a GUI, because the main focus of this coursework is algorithm implementation, data structures, and object-oriented design. The integrated program can be run through Main.java, while TaskAMain.java is kept as a separate entry point for testing Task A only. This makes the project easier to test and also easier to explain in the report and presentation.

The overall workflow of the application is shown below:

```
Read candidates_A.csv, candidates_B.csv, candidates_C.csv
→ Run Bubble Sort, Quick Sort, and Merge Sort
→ Select Top 10 locations from each dataset
→ Read paths.csv and build the weighted graph
→ Create the required inspection cases
→ Run shortest-path calculation
→ Print sorting results and path results
```

3.2 Data Structures Used

The program uses different data structures for different parts of the system. The choice of data structures is important because the project includes both sorting and graph shortest-path computation.

Program Part	Data Structure	Reason for Use
Candidate locations	ArrayList<Location>	Stores the candidate locations loaded from CSV files. It is convenient for sorting and for selecting the first 10 locations after sorting.
Individual location	Location object	Keeps location_id and priority_score together, which is clearer than using separate arrays.
Sorting result	SortingResult object	Stores the dataset name, algorithm name, average

		running time, and Top 10 locations in one object.
Selected targets	ArrayList<Location> / grouped selected lists	Keeps the selected Top 10 locations from Dataset A, B, and C so that Task B can reuse them.
Weighted graph	Adjacency list inside Graph	Stores only existing connections from paths.csv, which is suitable for graph traversal and shortest-path computation.
Graph edge	Edge object	Represents one weighted edge with a target node and a weight.
Path query case	InspectionCase object	Stores the start node, destination node, and required waypoint information for each required case.

For Task A, ArrayList<Location> is a simple and suitable choice because each candidate dataset contains 1000 locations. The program needs to sort the list directly and then take the first 10 elements after sorting. This matches the required workflow well.

For Task B, the graph is stored using an adjacency-list style design. This is more suitable than an adjacency matrix for this project because the program does not need to store every possible pair of locations. It only needs to store the real paths given in paths.csv. This saves space and also makes it easier to visit the neighbours of a node during Dijkstra's algorithm.

3.3 Main Classes and Responsibilities

The project is divided into several classes. Each class has a clear responsibility, which makes the program easier to read, test, and explain.

Class	**Main Responsibility**
Main	The entry point for running the complete integrated system.
TaskAMain	A separate entry point for running Task A only, mainly used for testing and collecting sorting output.
InspectionSystem	The central controller of the program. It runs Task A, prepares selected targets, builds the graph, and runs Task B.
InspectionCase	Represents one required shortest-path query case, including start node, destination node, and waypoint information.
Location	Stores one candidate location's ID and priority score.
CSVReader	Reads candidate CSV files and converts each row into a Location object.
Sorter	Defines the common sorting interface and shared ranking rule.
BubbleSort	Implements Bubble Sort using the shared ranking rule.
QuickSort	Implements Quick Sort using the shared ranking rule.
MergeSort	Implements Merge Sort using the shared ranking rule.
SortingResult	Stores the result of one sorting experiment, including average time and Top 10 locations.
Edge	Represents one weighted edge in the graph.

Graph	Stores the weighted graph and provides shortest-path and waypoint-based path query methods.

The design is not simply putting all methods into one file. Each class represents a meaningful part of the system. For example, the sorting classes do not need to know how the graph is built, and the graph class does not need to know how the candidate locations were sorted. InspectionSystem connects these parts and controls the whole workflow.

3.4 Integration of Task A and Task B

The most important part of Task C is the integration between Task A and Task B. Task B depends on Task A because the shortest-path cases use selected inspection targets from the Top 10 locations. Therefore, the program is designed so that the selected locations from Task A can be reused directly in Task B.

In the integrated system, InspectionSystem first runs the sorting process. It reads candidates_A.csv, candidates_B.csv, and candidates_C.csv, applies the sorting algorithms, and obtains the Top 10 locations from each dataset. After that, these selected locations are used to build the required path cases. For example, Case 2 uses the first and tenth selected locations from Dataset A. Case 3 uses the first selected location from Dataset A, the first selected location from Dataset B, and the fifth selected location from Dataset B. Case 4 uses the first selected location from Dataset A as the start node, the first selected location from Dataset C as the destination node, and the fifth selected locations from Dataset B and Dataset C as required waypoints.

This design is better than manually copying the selected locations into a separate graph program. It reduces the chance of using the wrong node ID and shows that the application works as one complete system instead of two isolated tasks.

3.5 Object-Oriented Design Principles

3.5.1 Encapsulation

Encapsulation is used by storing related data inside classes. For example, Location stores a location ID and its priority score together. Edge stores the information of one weighted graph edge. SortingResult stores the output of one sorting experiment. This is clearer than passing many separate variables around the program.

This also makes the code easier to understand. A Location represents one candidate inspection location, an Edge represents one road connection, and an InspectionCase represents one path query required by the coursework.

3.5.2 Abstraction

Abstraction is mainly shown through the Sorter interface. The program does not treat Bubble Sort, Quick Sort, and Merge Sort as completely unrelated pieces of code. Instead,

all three algorithms follow the same interface and provide their own implementation of sorting behaviour.

The Sorter interface also centralizes the comparison rule: higher priority_score comes first, and if the scores are equal, the smaller location_id comes first. This is useful because all three algorithms must follow exactly the same ranking rule. If the comparison logic were written separately in every sorting class, it would be easier to create inconsistent results.

3.5.3 Polymorphism

Polymorphism is used when the program runs different sorting algorithms through the same Sorter type. BubbleSort, QuickSort, and MergeSort all implement Sorter, so the program can store them together and call the same sort method on each of them.

This makes the sorting experiment easier to extend. If another sorting algorithm were added later, it could implement the same Sorter interface and be added to the list of algorithms with only small changes.

3.5.4 Separation of Responsibilities

The program also follows separation of responsibilities. CSVReader is responsible for reading input files. The sorting classes are responsible for sorting candidate locations. Graph is responsible for graph storage and shortest-path calculation. InspectionSystem is responsible for controlling the overall process.

This is better than placing all logic inside Main.java. If all file reading, sorting, graph construction, and path calculation were placed in one large main method, the program would be harder to debug and explain. With separate classes, each part can be checked more easily.

3.6 Design of the Graph Part

The graph part is mainly designed around the Graph and Edge classes. Each edge from paths.csv is stored as an Edge object with a target node and a weight. Since the graph represents an undirected road network, each connection should be treated as travelable in both directions.

The program uses Dijkstra's algorithm to calculate shortest paths because the graph is weighted and the edge weights represent distance or cost. This matches the coursework requirement. For cases with required waypoints, the program divides the route into several smaller shortest-path segments and then combines them. For example, a route from A1 to C1 via B5 and C5 can be handled as:

```
A1 → B5
B5 → C5
C5 → C1
```

The total path cost is then calculated by adding the cost of each segment. This design is easy to understand and matches the requirement that the path must pass through the waypoints in a fixed order. This approach solves the required fixed-order waypoint cases, although it does not try to solve a general travelling-salesman-style route planning problem.

3.7 Class Relationship Diagram

The following UML class diagram shows the main classes, interfaces, and relationships in the integrated application.

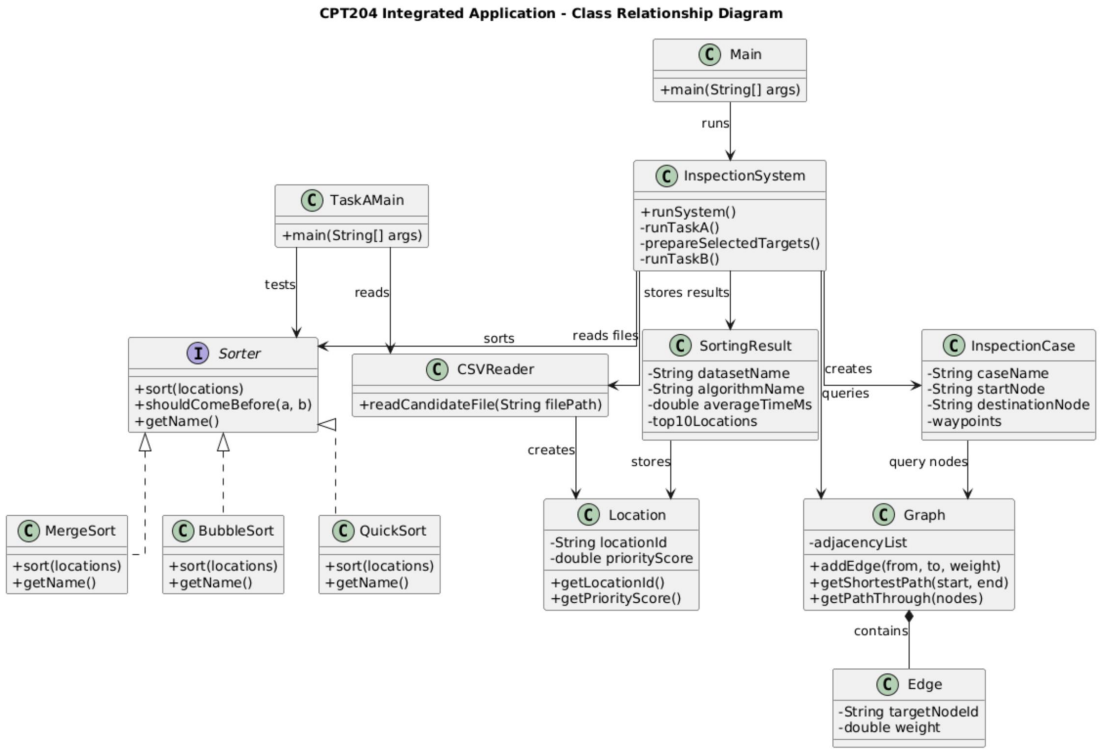

Figure 3.1. UML class diagram of the integrated application..

As shown in Figure 3.1, Main is the entry point of the full system, while InspectionSystem controls the overall workflow. It connects Task A and Task B by reading candidate datasets, running the sorting algorithms, preparing selected targets, building the graph, and executing the required path cases.

The Sorter interface is implemented by BubbleSort, QuickSort, and MergeSort. This shows abstraction and polymorphism, because the program can use the same Sorter type to run different sorting algorithms. CSVReader creates Location objects from the candidate files, and SortingResult stores the Task A output, including the algorithm name, average time, and Top 10 locations.

For Task B, Graph stores the weighted road network and contains Edge objects. InspectionCase stores the information for each required shortest-path query, such as the start node, destination node, and waypoints. TaskAMain is kept as a separate entry point for testing Task A only.

Overall, the UML diagram shows a clear separation of responsibilities. The program avoids putting all logic into Main, and instead uses different classes for reading data, sorting, storing results, graph representation, and workflow control. This also made debugging easier during development, because each problem could be checked in a smaller part of the program.

3.8 Design Strengths and Limitations

One strength of the design is that the workflow is clear. Task A and Task B are separated in logic, but they are still connected through InspectionSystem. This makes the program easier to run as a complete system. Another strength is that the sorting rule is centralized in the Sorter interface, which reduces the risk of inconsistent sorting results.

The project also provides two entry points. Main runs the full integrated system, while TaskAMain runs only Task A. This is useful during development because the sorting part can be tested separately without running all graph cases every time.

There are also some limitations. First, the file paths rely on the project root directory as the working directory. If the program is run from the wrong folder, it may not find the files inside the data folder. Second, the current program is console-based and does not include a graphical interface. This is acceptable for the coursework, but a real inspection system would probably need a clearer user interface. Third, the CSV reading logic assumes that the input files follow the expected format, so stronger validation would be needed in a larger application.

Overall, the current design is suitable for this coursework. It uses object-oriented classes to represent the main concepts, connects sorting and graph algorithms into one workflow, and keeps the code structure clear enough to explain, test, and maintain.